



University of Engineering & Technology Peshawar

Computer Programming (C++) Lab

Lab 09

STUDENT NAME	Muhammad Sahal Ayub
REGISTRATION NO:	4042F24
COUSRE:	EE-170L COMPUTER PROGRAMMING LAB
DATE:	28/04/26
Lab Title:	Introduction to Function in C++

SUBMITTED TO ENGR RIZWAN ULLAH

Objectives

By the end of this lab, students were able to:

- Understand what functions are and why they are used in C++.
- Declare function prototypes before the main() function.
- Define and call functions with zero, one, and multiple parameters.
- Work with functions that return values and those that do not (void).

Background / Theory

Functions allow a long program to be divided into named, reusable sections. A C++ function has a name, may accept parameters, and may return a result. The prototype of a function declares its return type, name, and parameter types so the compiler recognises calls before seeing the full definition.

The general syntax of a function prototype is:

```
returnType functionName(paramType1, paramType2, ...);
```

Functions we have used throughout the course, such as main(), follow this same pattern. void before a function name means it returns nothing; int, float, etc. mean it returns a value of that type.

Task 1: Function Introduction and Declaration

Problem Statement

Declare a function called "greet" that takes no parameters and returns nothing. Inside the function, display a greeting message of your choice. Call the function from the main program.

Source Code

```
#include <iostream>
using namespace std;

// Function prototype: greet takes no parameters and returns nothing
void greet();

int main()
{
    // Calling the greet function from main
    greet();
    return 0;
}

// Function definition: displays a custom greeting message
void greet()
{
    cout << "Welcome to EE-170L Computer Programming Lab!" << endl;
    cout << "We are learning Functions in C++ today." << endl;
}
```

Explanation

- The prototype void greet(); tells the compiler about the function before main().

- void means the function returns no value; the empty parentheses mean no parameters.
- greet() is called inside main() — execution jumps to the function body, prints the messages, then returns to main().

Expected Output

```
Welcome to EE-170L Computer Programming Lab!  
We are learning Functions in C++ today.
```

Task 2: Basic Function Declaration and Call

Problem Statement

Declare a function called "sayHello" that takes no parameters and returns nothing. Inside the function, display the message "Hello, World!". Call the function from the main program.

Source Code

```
#include <iostream>  
using namespace std;  
  
// Function prototype: sayHello takes no parameters and returns nothing  
void sayHello();  
  
int main()  
{  
    // Calling the sayHello function from main  
    sayHello();  
    return 0;  
}  
  
// Function definition: prints the classic Hello World message  
void sayHello()  
{  
    cout << "Hello, World!" << endl;  
}
```

Explanation

- sayHello() has an empty parameter list and void return type — identical in structure to greet() in Task 1.
- This task demonstrates the most basic form of a user-defined function.
- The function call in main() transfers control to the function body, which executes cout and then returns.

Expected Output

```
Hello, World!
```

Task 3: Array Sum and Average Using a Function

Problem Statement

Write a C++ program that takes 10 integers from the user into a 1D array, uses a function `calculateSum()` to find the sum, and then calculates and displays the average in `main()`.

Source Code

```
#include <iostream>
using namespace std;

// Function prototype: accepts array and its size, returns the sum
int calculateSum(int arr[], int size);

int main()
{
    const int SIZE = 10;           // Size of the array
    int numbers[SIZE];             // 1D array to store 10 integers
    double average;                // Variable to store the average

    // Taking 10 integer inputs from the user
    cout << "Enter 10 integer values:" << endl;
    for (int i = 0; i < SIZE; i++)
    {
        cout << "Element [" << i + 1 << "]: ";
        cin >> numbers[i];
    }

    // Calling calculateSum function and storing the result
    int sum = calculateSum(numbers, SIZE);

    // Calculating the average using the returned sum
    average = (double)sum / SIZE;

    // Displaying the results
    cout << "Sum of all elements: " << sum << endl;
    cout << "Average of all elements: " << average << endl;

    return 0;
}

// Function definition: iterates through array and accumulates sum
int calculateSum(int arr[], int size)
{
    int total = 0;                 // Variable to accumulate sum
    for (int i = 0; i < size; i++)
    {
        total += arr[i];           // Add each element to total
    }
    return total;                  // Return the final sum
}
```

Explanation

- The array `numbers[SIZE]` is declared in `main()` and filled via a for loop with user input.
- `calculateSum(int arr[], int size)` accepts the array and its size, iterates through all elements, and returns their total.
- `main()` calls the function, stores the returned sum, then computes the average as a double to preserve decimal precision.

- Using (double)sum casts the integer to a double before division, preventing integer truncation.

Sample Output

```
Enter 10 integer values:  
Element [1]: 5  
Element [2]: 10  
...  
Element [10]: 15  
Sum of all elements: 100  
Average of all elements: 10
```

